

Algorytmy i Struktury Danych

Lista 3

1 Wstęp

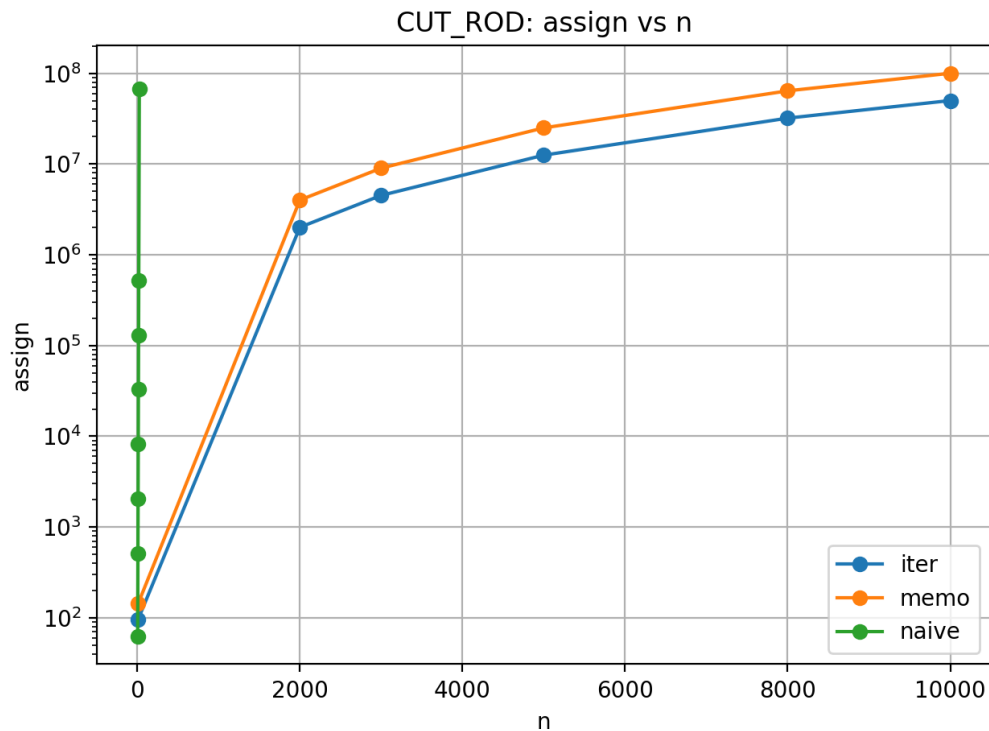
W ramach zadania zaimplementowano oraz przetestowano kilka algorytmów.

Dla algorytmów wykonano testy mierzące liczbę porównań (`cmp`) oraz przypisań (`assign`) w zależności od rozmiaru danych wejściowych. Wyniki zaprezentowano na wykresach.

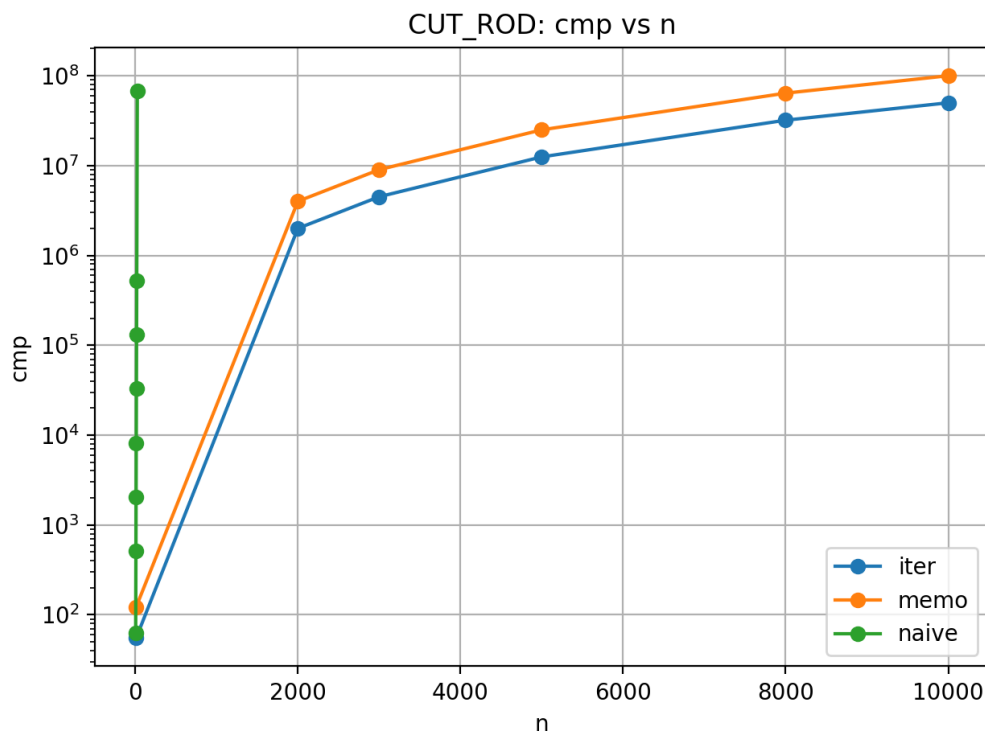
2 CUT_ROD

Problem CUT_ROD był testowany w trzech wersjach: naiwnej, ze spamiętywaniem oraz iteracyjnej. Już na etapie generowania danych było widać, że wersja naiwna bardzo szybko przestaje być użyteczna. Już dla rozmiaru 30 nie dałem rady doczekać na wynik. Na wykresach maksymalnie przedstawiono przypadek $n=20$.

2.1 Wyniki



Rysunek 1: CUT_ROD: liczba przypisań



Rysunek 2: CUT_ROD: liczba porównań

Zastosowano skalę logarytmiczną, ponieważ wersja naiwna rośnie wykładniczo i bardzo szybko „ucieka” na wykresach w porównaniu do pozostałych wersji.

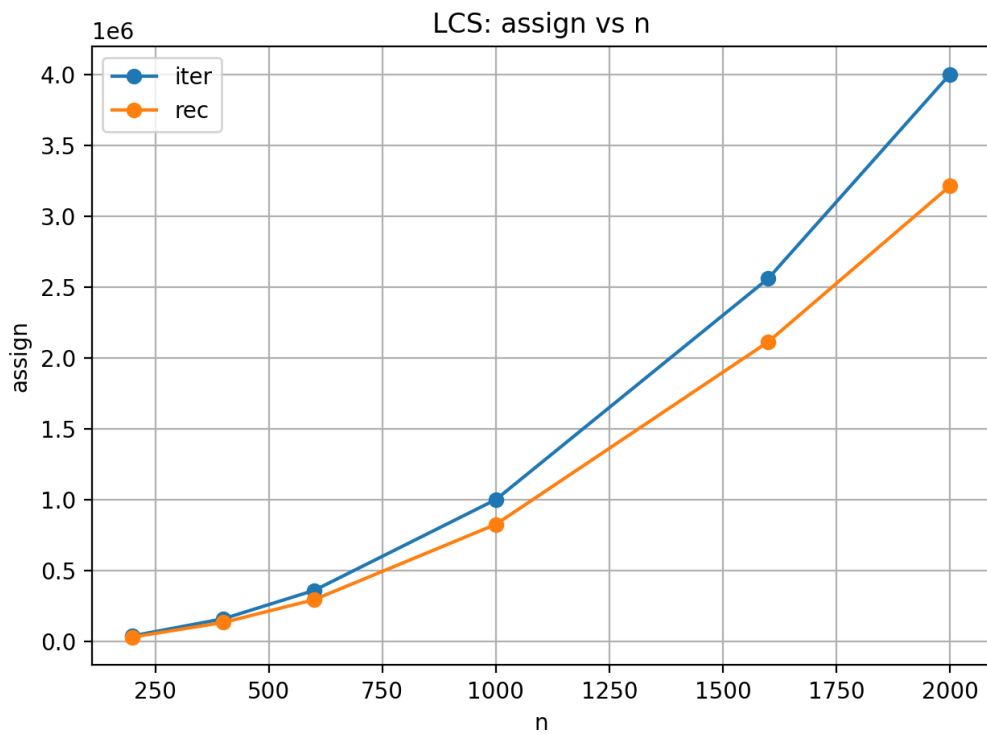
2.2 Komentarz

Wersje ze spamiętywaniem oraz iteracyjna zachowują się stabilnie i rosną w przewidywalny sposób. Wersja naiwna działa poprawnie tylko dla bardzo małych danych i ma głównie znaczenie teoretyczne. Ciekawe jest, że wersja iteracyjna zachowuje się trochę lepiej niż wersja ze spamiętywaniem, co może wynikać z mniejszej ilości wywołań funkcji i związanych z tym narzutów.

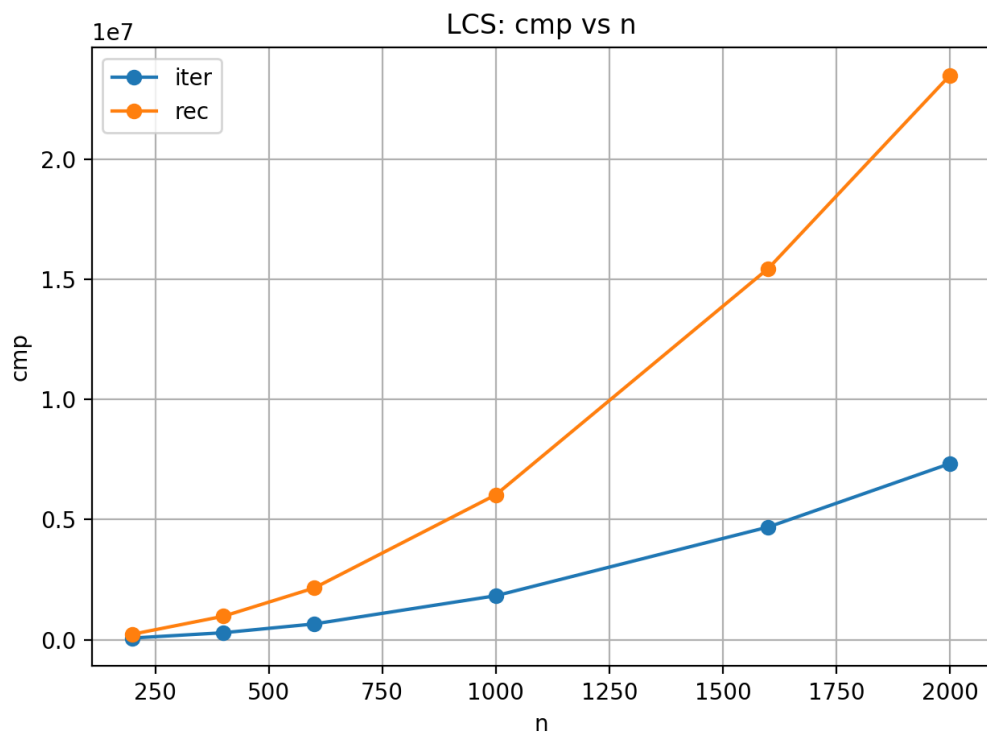
3 LCS

Algorytm LCS został zaimplementowany w wersji rekurencyjnej ze spamiętywaniem oraz iteracyjnej. Obie wersje mają złożoność kwadratową, dlatego różnice pomiędzy nimi są mniejsze niż w przypadku CUT_ROD.

3.1 Wyniki



Rysunek 3: LCS: liczba przypisań



Rysunek 4: LCS: liczba porównań

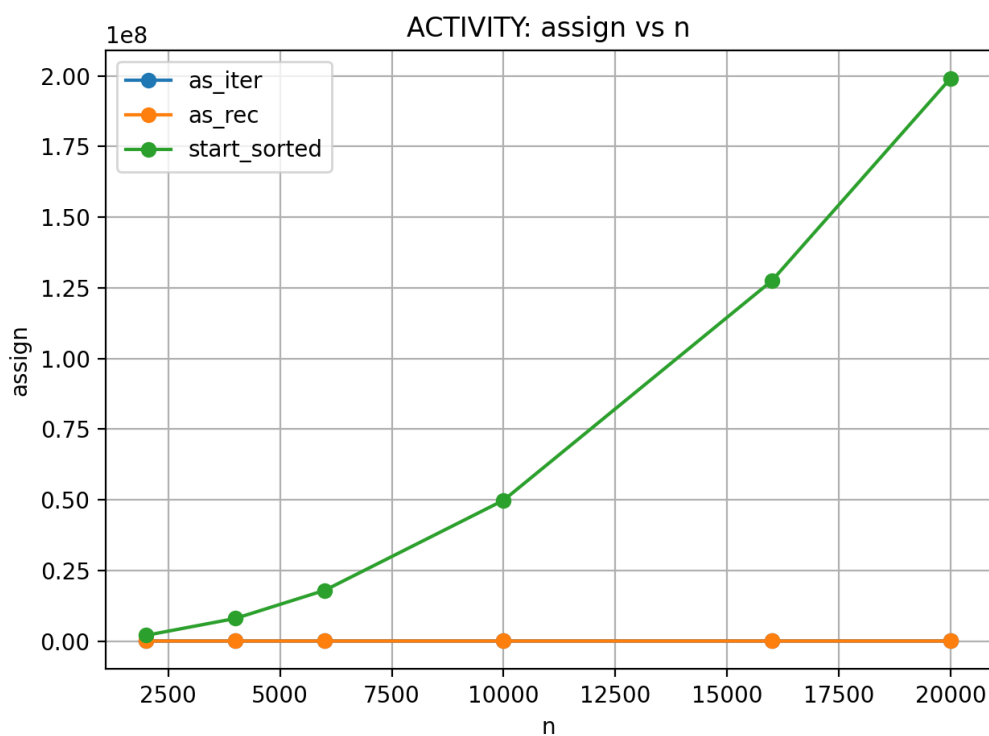
3.2 Komentarz

Wersja iteracyjna wykonuje mniej operacji niż wersja rekurencyjna, jeśli liczymy sumę przypisywań i porównań, co jest widoczne szczególnie dla większych danych. Ciekawe jest natomiast to, że w zależności od tego, co jest w danym momencie ważniejsze, można wybrać odpowiednią wersję. Różnice nie są jednak bardzo duże, ponieważ oba algorytmy rosną zgodnie z oczekiwaną złożonością $O(n^2)$.

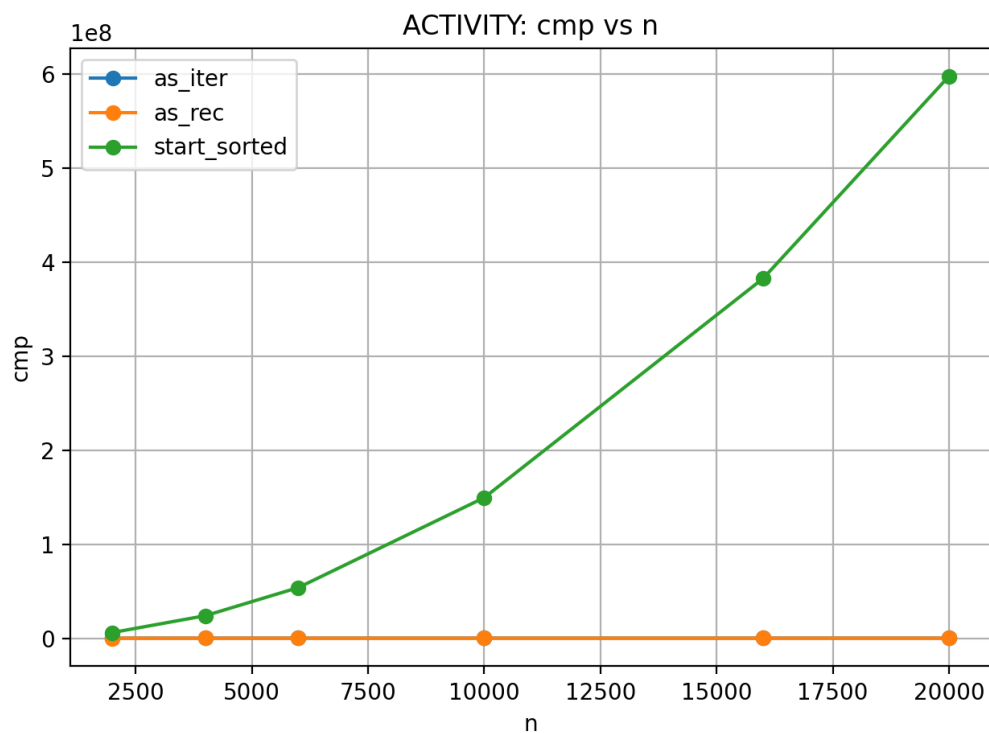
4 ACTIVITY_SELECTOR

Dla problemu wyboru aktywności zaimplementowano trzy wersje: rekurencyjną, iteracyjną oraz dynamiczną o złożoności kwadratowej.

4.1 Wyniki

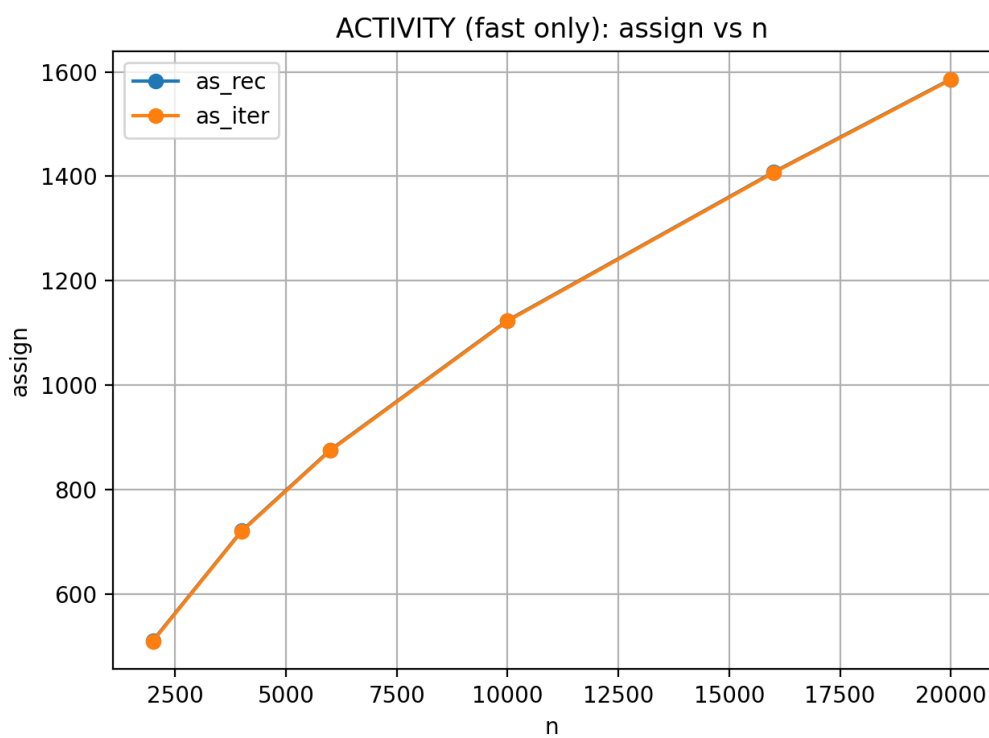


Rysunek 5: ACTIVITY: liczba przypisań

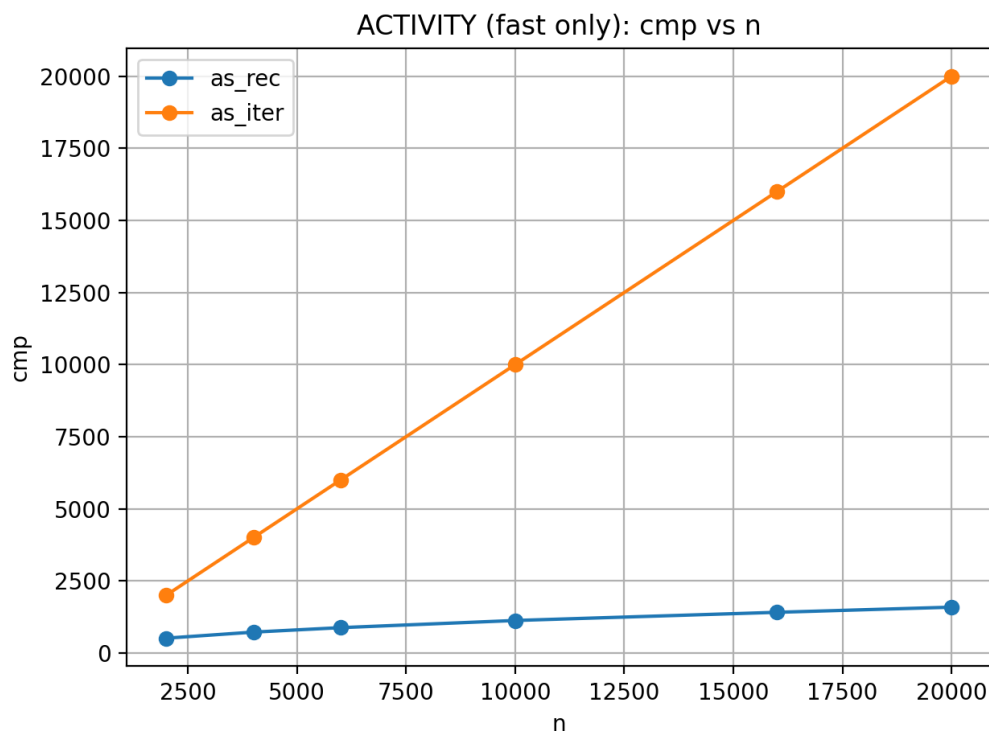


Rysunek 6: ACTIVITY: liczba porównań

Algorytm dynamiczny bardzo szybko zaczyna wychodzić poza wykres, bo ma złożoność kwadratową, dlatego dodatkowo pokazano porównanie tylko dwóch szybkich wersji.



Rysunek 7: ACTIVITY (szybkie wersje): liczba przypisań



Rysunek 8: ACTIVITY (szybkie wersje): liczba porównań

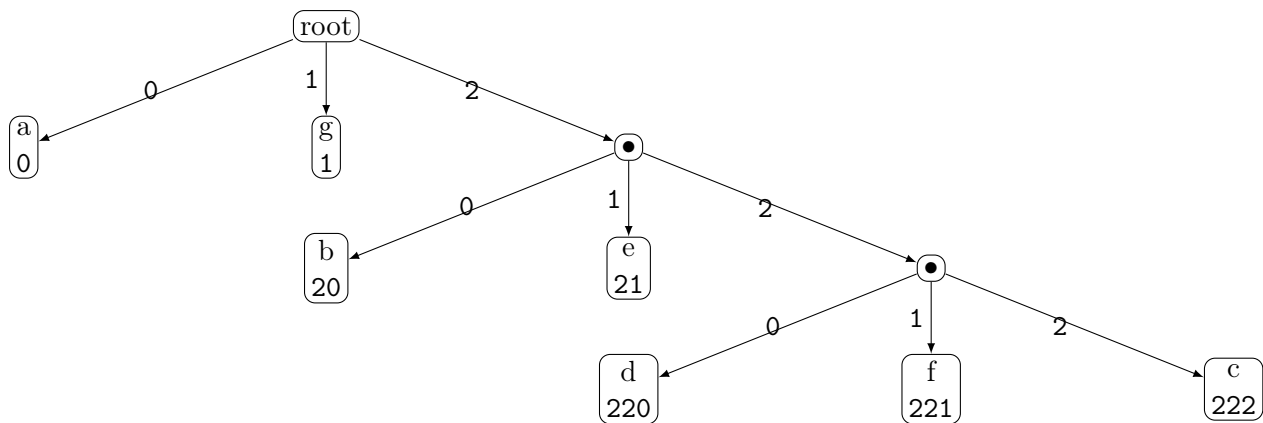
4.2 Komentarz

Rozwiązanie dynamiczne bardzo szybko staje się niepraktyczne. Ciekawe jest, że wersja rekurencyjna i iteracyjna zachowują się nawet nie podobnie, a identycznie w przypadku liczby przypisań, ale różnią się w liczbie porównań, gdzie wersja iteracyjna jest zauważalnie wolniejsza.

Ale chcę podkreślić, że wersja bazująca na zajęciach posortowanych według czasu rozpoczęcia jest mniej szybka dlatego, że taki algorytm jest sam w sobie bardziej złożony.

5 Algorytm Huffmana

Algorytm Huffmana w wersji ternarnej wydaje się być najbardziej interesującym elementem całego zadania. Jak najbardziej ciekawe było znalezienie idei z dodawaniem kilku elementów pustych, aby drzewo mogło być poprawnie zbudowane.



Rysunek 9: Drzewo Huffmana (ternarne) dla tekstu "aaaaaaabbbccdeeeeffggggggggg"

Liczba kodowanych symboli powinna być nieparzysta, bo w innym przypadku algorytm może zostawić pustą krawędź, co nie jest optymalne.

Algorytm ten był szczególnie ciekawy podczas implementacji i testów, ponieważ nawet dla krótkiego tekstu struktura drzewa i przypisane kody nie są oczywiste na pierwszy rzut oka.

6 Porównanie algorytmów

Algorytm	Szybkość	Komentarz
CUT_ROD (naiwny)	bardzo wolny	szybko przestaje mieć sens
CUT_ROD (iter lub memo)	szybki	stabilne zachowanie
LCS	szybki	rośnie kwadratowo
ACTIVITY (iter lub rec)	bardzo szybki	najlepszy wybór w praktyce
ACTIVITY (start sorted)	wolny	jeśli sortowanie danych jest kosztowne
Huffman	szybki	najbardziej „niezwykły” algorytm

7 Podsumowanie

Na podstawie wykresów i testów można zauważyć, że algorytmy naiwne lub kwadratowe bardzo szybko przestają być użyteczne dla większych danych. Zastosowanie spamiętywania lub iteracji daje znaczną poprawę wydajności.

Algorytm Huffmana wyróżnia się tym, że oprócz analizy złożoności pozwala na obserwację konkretnego efektu działania algorytmu, co czyni go najbardziej „wizualnym” elementem całego zadania.